Sprite, Surf and Rect

04 November 2025 20:20

```
import pygame
from pygame.locals import *
# Define our Square object and call super two. Give it all the properties and methods of
# pygame.sprite.Sprite.
# Define the class for our square objects.
class Square(pygame.sprite.Sprite):
    def __init__(self):
        super(Square, self).__init__()

        # Define the dimension of the surface.
        # Here we are making squares of side 25PX.
        self.surf = pygame.Surface((25,25))

        # Define the color of the surface using RGB color coding.
        self.surf.fill((0,200,255))
        self.rect = self.surf.get_rect()

#initialize pygame
pygame.init()
# Define the dimension of screen object.
screen = pygame.display.set_mode((800,600))
sq1 = Square()
sq2 = Square()
sq3 = Square()
sq4 = Square()
# Variable to keep our game loop running.
gameOn = True
# Our game loop.
while gameOn:
    # For loop through the event queue.
    for event in pygame.event.get():
        # Check the key down event.
        if event.type == KEYDOWN:
            # If the backspace key has been pressed, set.
            # Running two false to exit the main loop.
            if event.key == K_BACKSPACE:
                gameOn = False
            elif event.type == QUIT:
                gameOn = False

        # Define where the squares will appear on the screen.
        # Use blitz to draw them on the screen surface.
        screen.blit(sq1.surf,(40,40))
        screen.blit(sq1.surf,(40, 530))
        screen.blit(sq1.surf,(730,40))
        screen.blit(sq1.surf,(730, 530))

        #Update the display using flip
        pygame.display.flip()
```

we import pygame and pygame.locals which is necessary to do before using any module in Python's pygame

method is use to create Object of type Square.

25 25

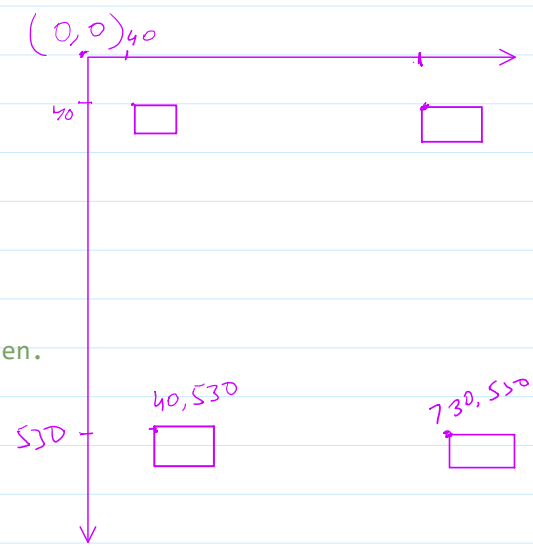
Sprite

Square

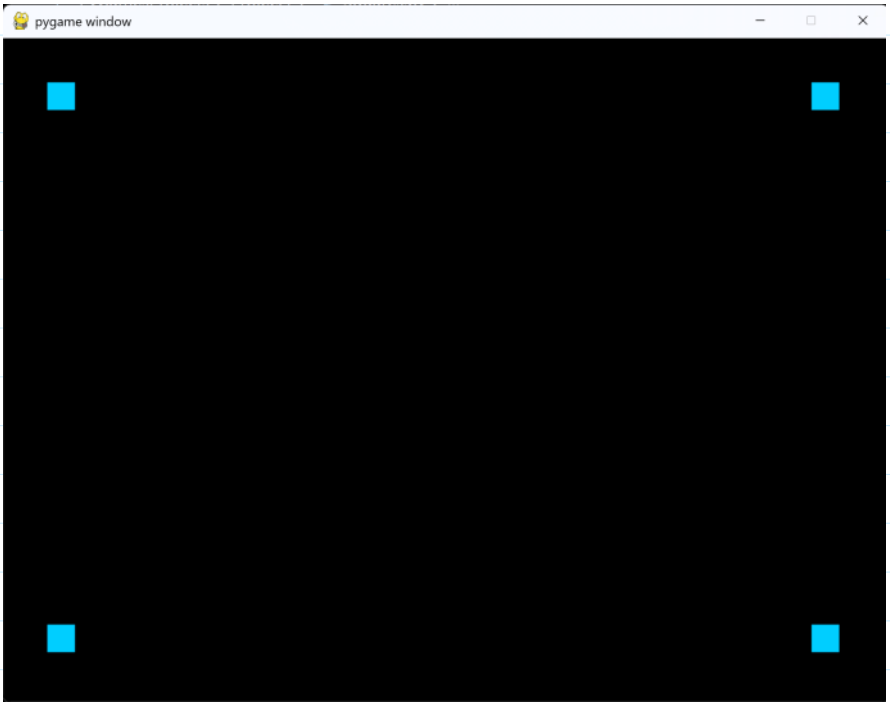
instance variable

cyan

4 objects of type Square, This method is calling __init__(self) method.



Output:



Sprite, Surf, and Rect

Sprite: Sprite is just a 2D object that we draw on the screen. We can use them by extending the Sprite class.

Surf: Surfaces like blank sheets of paper on which we draw. Power screen object is also a surface that can hold images as well.

Rects: Rectangular area that we define on a surface.

Sprite
"IS-A"
Square

```
class Square(pygame.sprite.Sprite):
    def __init__(self):
        super(Square, self).__init__()

        # Define the dimension of the surface.
        # Here we are making squares of side 25PX.
        self.surf = pygame.Surface((25,25))
                                    tuple

        # Define the color of the surface using RGB color coding.
        self.surf.fill((0,200,255))
                        tuple

        self.rect = self.surf.get_rect()
```

Blit and Flip-

Blit: Blitz keyword is used to draw a surface on another surface. In simple words, when we draw a surface we just blit it on to some other surface.

Flip: It is used to update the entire screen after everything is drawn. Remember that the flip only works after drawing all the necessary surfaces, otherwise it will update nothing.

Pygame is Python library designed for game development. PyGame is built on the top of SDL

library, so it provides full functionality to develop game in Python. Pygame has many modules to perform its operation. Before these modules can be used, they must be initialized. All the modules can be initialized individually or one at a time.

Methods used:

- **pygame.init():-** To initialize all the modules, it takes no argument and return a tuple (numpass, numfail) Which indicate the number of modules initialized successfully and the number of modules failed.
- **pygame.get_init():-** This method is used to check whether Pi game modules are initialized or not.

```
import pygame
(numpass, numfail)= pygame.init()
print('Number of modules initialized successfully:', numpass)
print('Number of modules haven\'t initialized successfully:', numfail)
```

Output:

```
Number of modules initialized successfully: 5
Number of modules haven't initialized successfully: 0
```

```
import pygame
pygame.init()
is_initialized = pygame.get_init()
print('Is pygame modules initialized:', is_initialized)
```

Output:

```
Hello from the pygame community. https://www.pygame.org/contribute.html
Is pygame modules initialized: True
```

Get the size of PyGame window

```
import pygame
pygame.init()
screen = pygame.display.set_mode()
x, y = screen.get_size()
pygame.display.quit()
print(x, y)
```

Output:

```
Hello from the pygame community.
1536 864
```

width height.

width height is modified.

```
import pygame
pygame.init()
screen = pygame.display.set_mode((500,500))
x, y = screen.get_size()
pygame.display.quit()
print(x, y)
```

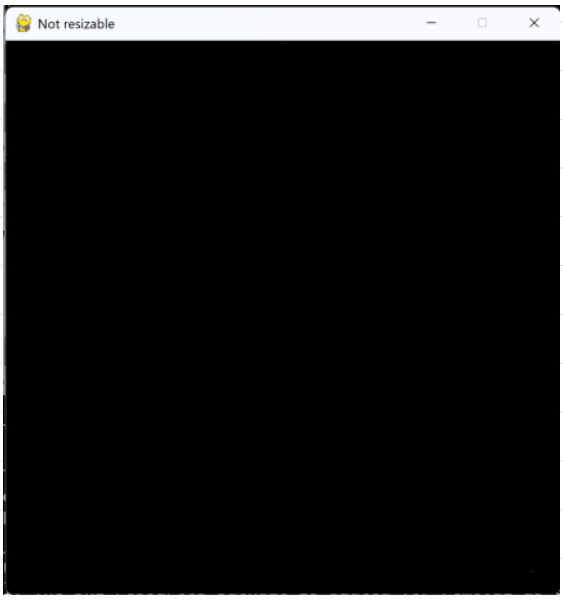
Output:

```
Hello from the pygame community.
500 500
```

Allowing resizing window in PyGame

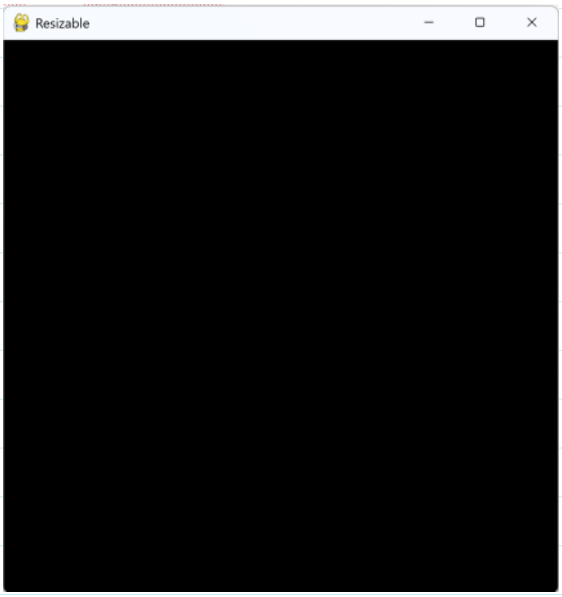
```
import pygame
pygame.init()
screen = pygame.display.set_mode((500,500))
pygame.display.set_caption('Not resizable')
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```

Output:




```
import pygame
pygame.init()
screen = pygame.display.set_mode((500,500), pygame.RESIZABLE)
pygame.display.set_caption('Resizable')
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
pygame.quit()
```

Output:



Change screen background color in PyGame

Pie game is a Python library designed to develop video games. Pygame adds functionality on top of the excellent SDL library. This allows you to create fully featured games and multimedia programs in the Python language.

Functions used:

- **pygame.init():** This function is used to initialize all the pygame modules.
- **pygame.display.set_mode():** This function is used to initialize a screen for display. And we can set the size of the window.
- **fill():** This method is used to fill the display with the color specified.

```
import pygame
pygame.init()
surface = pygame.display.set_mode((1000, 600))
color = (100, 25, 150)
surface.fill(color)
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

Changing the Name of a PyGame window

PyGame Window is a simple window that displays our game on the Window screen. By default pygame uses "pygame window" as its title and pygame icon as its logo for pygame window. We can use **set_caption()** function to change the name and **set_icon()** to set icon of our window.

To change the name of pygame window:

Syntax:

```
pygame.display.set_caption("Title of window")
```

To change the icon of pygame window:

Syntax:

```
pygame.display.set_icon("Icon_name")
```

Stepwise implementation (Algorithm):

1. First we import and initialize all imported modules. We use import pie game to import all modules and dot in it () function to initialize those modules.
```
import pygame
pygame.init()
```

2. Initialize a window to display. We use **.set_mode()** Function to create a window. We pass the width and height of our window as parameters to set_mode() function.
```
   pygame.display.set_mode((width, height))
```

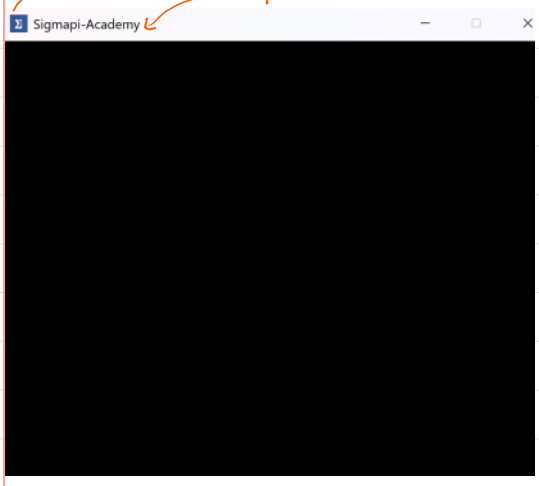
3. To change default title and icon of pygame window we use .set_caption() and .set_icon() functions. To change icon first we load icon image using pygame.image.load("image_path") function and then use .set_icon() to change default image.
```
   pygame.display.set_caption("Tic Tac Toe")
   Icon = pygame.image.load("icon_image with path")
   pygame.display.set_icon(Icon)
```

4. Keep that window running until the user presses the exit button. We use a variable that is true unless the user presses the quit button. To keep the game running we use a while loop and check our variable if it is true or not.

```
running = True
while running:
   for event in pygame.event.get():
      if event.type == pygame.QUIT:
         running = False
```

```
import pygame
pygame.init()
pygame.display.set_mode((500, 400))
pygame.display.set_caption("Sigmapi-Academy")
logo = pygame.image.load("PyGamesProjects/Project1/images/spa.png")
pygame.display.set_icon(logo)
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

logo caption.



Pygame - Surface

When using pygame, surfaces are generally used to represent the appearance of the object and its position on the screen. All the objects, text, images that we create in pygame are created using surfaces.

Creating a surface

Creating surfaces in pygame is quite easy. We just have to pass the height and the width of a tuple to pygame.Surface() method. We can use various methods to format our surface as we want. For example, we can use pygame.draw() to draw shapes, we can use surface.fill() method to fill on the surface. Now let's discuss the implementation of these functions:

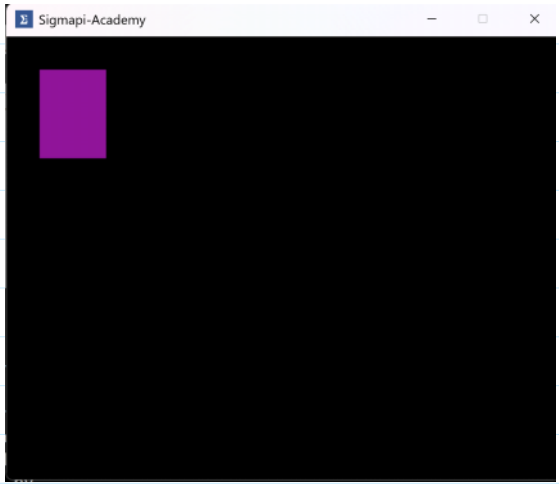
Syntax:

```
pygame.Surface((width, height, flags, depth, mask))

pygame.draw(): It is used to draw an object shape.

Surface.rect(surface, color, rect)
```

Code without surface.fill() method:

```
import pygame
pygame.init()
surface = pygame.display.set_mode((500, 400))
pygame.display.set_caption("Sigmapi-Academy")
logo = pygame.image.load("PyGamesProjects/Project1/images/spa.png")
pygame.display.set_icon(logo)
pygame.draw.rect(surface, (140,20,150), pygame.Rect(30,30,60,80))
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

Output:



Surface_name.fill(): It is used for filling color in the surface.

Syntax:

```
pygame.Surface.fill(color, rect = None, special_flags=0)
```

```
import pygame
pygame.init()
surface = pygame.display.set_mode((800, 500))
pygame.display.set_caption("Sigmapi-Academy")
logo = pygame.image.load("./PyGamesProjects/Project1/images/spa.png")
pygame.display.set_icon(logo)
```

```
surface.fill((0,0,0)) # background color.
pygame.draw.rect(surface, (140,20,150), pygame.Rect(30,30,60,80))
image = pygame.image.load("./PyGamesProjects/Project1/images/image.png")
surface.blit(image, (300,300))
pygame.display.flip()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

Output:

Home work:

1. Screen Background and Surface Management in PyGame

- Write a complete PyGame program to create a window of size 700x400 pixels. Change the background color to RGB (34, 139, 34). Create two rectangular surfaces of different colors and display them at different locations. Explain each step including surface creation, color filling, and drawing on the main window.

2. Window Title, Icon, and Custom Quit Events

- Create a PyGame window and set its title to "Practice Game". Use a custom icon image (size 30x30 px) for your window (you may use the "Sigmapi Academy" icon previously generated). Implement an event handler that closes the window when either the standard close button or the spacebar is pressed. Describe how you set the caption, icon, and manage multiple quit events.

3. Interactive Drawing and Dynamic Background Change

- Write a PyGame application that lets the user change the background color by pressing different keys (R for red, G for green, B for blue). Draw a shape (such as a circle or rectangle) that moves when arrow keys are pressed. Include comments and explain your approach to handling keyboard events and updating the display.

4. Loading and Blitting Multiple Images

- Develop a PyGame project where you load three different images (including "image.jpg" attached). Display them at different positions and sizes on the window. Change one image's position continuously in a loop to create a simple animation. Discuss the process of image loading, scaling, and blitting in PyGame.

5. Surface Drawing with Custom Functions

- Implement a PyGame program where you define and use custom functions to draw multiple

geometric shapes (rectangle, circle, polygon) on a surface. The user can press keys to toggle visibility of each shape. Include logic to clear and redraw the screen when the visibility changes, and explain how surfaces, fill, and draw calls can be managed efficiently.